SKILL ENHANCEMENT COURSE (SEC)

ANDROID PROGRAMMING

Official Skill Enhancement Course (SEC) Syllabus — Complete Topper Master Handbook

Exhaustive Theory + Java OOP Foundation + Eclipse ADT Environment + 7-Stage Activity Life Cycle + UI Layout Hierarchy + SQLite CRUD Database Application + Viva Questions & Exam Strategy.

Unit I	Unit II	Unit III	Unit IV	Unit V
OS & 5-Layer Stack	Java OOP & JVM	Eclipse ADT Setup	Activity & UI Layouts	SQLite CRUD DB

👨‍💻 Android Notes Authored & Curated by Aakash Rajawat

BCA Student · Computer Science & Systems Engineering Enthusiast

Profiles: 💻 GitHub (aakashsinghrajawat75) • 💼 LinkedIn (aakashrajawat75) • 📊 Kaggle (aakashrajawat75) • 🚀 GeeksforGeeks (aakashsinghrajawat)

COMPREHENSIVE SYLLABUS TABLE OF CONTENTS

Unit I: Introduction to Android

- History of Android & Version Timeline
- Android OS Overview & Key Features
- 5-Layer Architecture Stack Deep-Dive
- Dalvik Virtual Machine (DVM) vs JVM
- Android SDK Tools & ADB Commands

Unit II: Java OOP & JVM

- OOP Principles & Access Modifiers
- Inheritance & Polymorphism
- Interfaces vs Abstract Classes
- Multithreading & Thread Synchronization
- JVM Architecture & Garbage Collection
- 7 Essential Exam Java Programs

Unit III: Dev Tools & Setup

- Eclipse IDE + ADT Plugin Setup
- Android Virtual Device (AVD / Emulator)
- AndroidManifest.xml & Project Structure
- Hello World XML Layout & Activity
- Deploying on USB Hardware Device

Unit IV: UI Design & Lifecycle

- Application Context & Intent Passing
- Activity Life Cycle (7 States & Transitions)
- Screen Sizes, Densities & dp/sp Math
- Layout Managers (Linear, Relative, Table, Frame)
- UI Form Controls & Event Listeners
- Menus, Dialogs & Toast Notifications

Unit V: Database (SQLite)

- SQLite Characteristics & Data Types
- SQLiteOpenHelper Class Mechanics
- Complete Student CRUD Java Application
- Application to SQLite Architecture Flow

Exam Master Section

- 12 Comparison Master Tables
- 25 Important Definitions
- 2M, 5M & 10M Question Bank
- Topper Exam Writing Strategy

I

Unit I: Introduction to Android & Software Stack Architecture

Core Fundamentals · Weightage: 20-25 Marks

> 📌 **Unit Syllabus Overview & Learning Objectives**

1.1 History of Android & Version Evolution ⭐ Important

> **Origin & Formation of Android**
>
> - **Founded:** October 2003 in Palo Alto, California by **Andy Rubin**, Rich Miner, Nick Sears, and Chris White. Originally intended as an advanced OS for digital cameras before pivoting to mobile smartphones.
> - **Google Acquisition:** Acquired by **Google Inc.** in August 2005. Andy Rubin and team continued development under Google.
> - **Open Handset Alliance (OHA):** Formed on **November 5, 2007**. A consortium of 84 telecom and tech leaders (Google, HTC, Sony, Dell, Intel, Motorola, Qualcomm, Samsung) committed to open standards for mobile devices.
> - **First Commercial Android Device: HTC Dream (T-Mobile G1)** released in October 2008 running Android 1.0.

Android Version Evolution & Code Names Timeline:

Android versions were famously named alphabetically after desserts and sweet treats (from Cupcake to Pie) until numeric naming started from Android 10:

Version	Code Name	API Level	Release Year	Key Technological Milestones
1.5	**Cupcake**	API 3	2009	On-screen virtual software keyboard, video recording/playback, third-party App Widgets.
1.6	**Donut**	API 4	2009	Quick Search Box, support for multiple screen resolutions (WVGA/QVGA), Text-to-Speech (TTS) engine.
2.0 / 2.1	**Eclair**	API 5-7	2009	Google Maps turn-by-turn navigation, HTML5 browser support, multi-

Version	Code Name	API Level	Release Year	Key Technological Milestones
				account syncing, live wallpapers.
2.2	**Froyo** (Frozen Yogurt)	API 8	2010	Dalvik **JIT (Just-in-Time) compiler** (2x-5x speed boost), USB tethering, Wi-Fi hotspot functionality.
2.3	**Gingerbread**	API 9-10	2010	NFC (Near Field Communication), native gaming APIs, improved copy-paste, front-facing camera support.
3.0 - 3.2	**Honeycomb**	API 11-13	2011	Tablet-only specialized UI, Action Bar, Fragment architecture, multi-core CPU optimizations.
4.0	**Ice Cream Sandwich** (ICS)	API 14-15	2011	Unified Smartphone & Tablet UI, Roboto font, resizable home-screen widgets, Face Unlock, Wi-Fi Direct.
4.1 - 4.3	**Jelly Bean** (Syllabus Target)	**API 16-18**	2012-13	**Project Butter (smooth 60fps VSYNC UI rendering), Google Now predictive assistant, expandable notifications, Bi-directional text support.**
4.4	**KitKat**	API 19	2013	Optimized to run on entry-level 512 MB

Version	Code Name	API Level	Release Year	Key Technological Milestones
				RAM devices ("Project Svelte"), Chromium webview, full-screen immersive mode.

> 💡 **Modern Android Development Bridge (Academic Syllabus vs. Industry Context)**
>
> **Note for Competitive Exams & Technical Interviews:** While the BCA Semester V syllabus emphasizes foundational Android concepts (**Java OOP**, **Eclipse ADT Plugin**, **Dalvik Virtual Machine (DVM)**, **Android 4.x Jelly Bean**, and **Raw SQLiteOpenHelper**), the modern production industry (2024–2026+) has evolved significantly:
>
> - **IDE:** *Android Studio* (powered by JetBrains IntelliJ IDEA with Gradle Build System) has completely replaced Eclipse ADT since 2015.
> - **Programming Language:** *Kotlin* is now Google's official first-class language for Android, offering null-safety, concise syntax, and Kotlin Coroutines/Flow for reactive asynchronous programming.
> - **Declarative UI:** *Jetpack Compose* has largely superseded imperative XML layout files with reactive, composable Kotlin functions.
> - **Runtime Execution:** *Android Runtime (ART)* with Ahead-Of-Time (AOT) and profile-guided compilation has replaced Dalvik VM for faster app launches and smoother 120Hz UI rendering.
> - **Data Persistence:** *Room Persistence Library* (part of Android Jetpack) provides an abstraction layer and ORM over SQLite with compile-time query validation.

1.2 Android Operating System Overview & Key Features 🔥 Very Important

> 📌 **Formal Academic Definition**

Core Salient Features of Android OS:

1. **Open-Source & Free Licensing:** Published under the permissive *Apache 2.0 License* (and GNU GPL for the Linux kernel). Handset makers can freely customize the operating system for their hardware.
2. **Multitasking & Concurrent Process Isolation:** Each Android application runs in its own isolated Linux process with a dedicated sandboxed virtual machine instance, ensuring that a crashing app cannot bring down the operating system.
3. **SQLite Relational Database:** Integrated lightweight, zero-configuration relational database engine embedded directly in the OS for structured local offline data persistence.
4. **Rich Connectivity & Networking:** Native platform support for GSM/EDGE, IDEN, CDMA, EV-DO, UMTS, LTE, 5G, Wi-Fi, Bluetooth, NFC, and Ultra-Wideband (UWB).
5. **Unified Hardware & Sensor Abstraction:** Standardized APIs to access onboard hardware peripherals: Multi-touch screen, Accelerometer, Gyroscope, Magnetometer (Compass), GPS, Barometer, Ambient Light Sensor, Proximity Sensor, and Biometric scanners.
6. **Optimized Virtual Machine (DVM / ART):** Custom execution engines engineered specifically for battery-powered, memory-constrained embedded mobile devices.

7. **Rich 2D & 3D Graphics Subsystem:** Hardware-accelerated graphics using Skia 2D graphics engine and OpenGL ES 2.0/3.0 / Vulkan for high-performance 3D graphics and mobile gaming.
8. **Comprehensive Media Support:** Built-in codecs for audio/video playback and recording: MP3, AAC, AMR, MIDI, Ogg Vorbis, WAV, JPEG, PNG, GIF, BMP, WebP, MPEG-4, H.264, and VP8/VP9.

1.3 Android 5-Layer Software Stack Architecture 🔥 10-Mark Guaranteed Question

Architecture Stack Overview

📊 Layered Diagram: Complete Android 5-Layer Architecture Stack

Exhaustive Breakdown of the 5 Layers:

Layer 1: Applications Layer (Top Tier)

The topmost user-facing layer containing applications written in Java. Divided into two categories:

- **Native Core System Apps:** Shipped out-of-the-box by Google/OEM (Home Launcher, Contacts, Phone Dialer, SMS Messaging, Browser, Email, Camera, Settings, Clock).
- **Third-Party Applications:** Installed by users from Google Play Store or sideloaded APKs (WhatsApp, Instagram, Custom BCA Project Apps).

Layer 2: Application Framework Layer

Provides the high-level reusable Java classes, managers, and service architectures that developers use to build applications:

- **Activity Manager:** Controls the activity lifecycle, back stack navigation, and process states.
- **Window Manager:** Manages top-level window displays, surfaces, and screen orientation.
- **Content Providers:** Encapsulates and shares persistent data securely across independent applications (e.g., querying device Contacts database).
- **View System:** Rich hierarchy of extensible UI components (Buttons, TextViews, EditBoxes, Layouts, Lists, Grids) that handle UI rendering and touch event dispatching.
- **Notification Manager:** Allows applications to display custom status-bar alerts, badges, and heads-up notifications.
- **Package Manager:** Queries information regarding installed APK packages and permissions on the device.
- **Resource Manager:** Provides non-code resources (XML layouts, localized string translations, graphic drawables, UI color palettes).
- **Location Manager:** Provides periodic location updates via GPS, Wi-Fi cell tower triangulation, and network location providers.
- **Telephony Manager:** Manages voice call states, SIM card details, network operator info, and signal strength.

Layer 3: C/C++ Native Libraries & Android Runtime (ART / Dalvik)

Contains two parallel side-by-side modules:

- **Native C/C++ Libraries:** High-performance compiled binaries that handle heavy multimedia, database, and rendering computation:
  - *SurfaceFlinger:* System-wide graphics compositor that renders 2D/3D visual layers to the display screen buffer.

• *SQLite:* Embedded SQL database engine for structured local data persistence.
• *WebKit / Chromium:* Modern web rendering engine used to power the Android Browser and embedded `WebView` controls.
• *Media Framework:* PacketVideo/OpenCORE based framework for hardware-accelerated audio/video recording and playback.
• *OpenGL ES:* Cross-language 3D graphics rendering acceleration.
• *FreeType:* Bitmapped and vector font rendering engine.
• *Bionic libc:* Custom BSD-derived C standard library tuned for low memory footprints and fast execution on embedded Linux.

- **Android Runtime (ART / Dalvik):**
• *Core Java Libraries:* Java standard libraries (Data structures, I/O, Networking, Utilities).
• *Dalvik Virtual Machine (DVM):* Register-based execution engine that executes compact Dalvik Executable (`.dex`) bytecodes. Each app runs inside its own DVM instance.

Layer 4: Hardware Abstraction Layer (HAL)

Provides standardized C/C++ interface hooks that bridge the gap between high-level Java API framework calls and low-level Linux Kernel hardware device drivers. HAL ensures that Google can update the Android OS without device manufacturers (Qualcomm, MediaTek, Samsung) having to rewrite low-level drivers.

Layer 5: Linux Kernel Layer (Base Tier)

The foundation of the entire software stack. Android uses a customized Linux Kernel (version 2.6 up to 5.x) providing:

- **Hardware Device Drivers:** Display, Camera, Flash Memory, Bluetooth, Audio, Keypad, Wi-Fi, and Power Drivers.
- **Binder IPC Driver:** High-speed custom inter-process communication driver allowing safe data transfer between different application sandboxes.
- **Memory Management & Low Memory Killer (LMK):** Aggressively terminates background cached processes when physical RAM is exhausted.
- **Power Management (Wakelocks):** Prevents CPU sleep during critical background playback or download tasks.
- **Process Isolation & Security UID:** Assigns a unique Linux User ID (UID) to every installed application, completely sandboxing its file system from other apps.

1.4 Dalvik Virtual Machine (DVM) vs Java Virtual Machine (JVM) 🔥 Very Important

Why DVM was created for Mobile Devices

📊 Visual Diagram: Java JVM vs Android Dalvik (DVM) Compilation Pipeline

Exhaustive Comparison: JVM vs DVM

Feature / Property	Standard JVM (Java Virtual Machine)	Android DVM (Dalvik Virtual Machine)
Bytecode Executable	Executes Java Bytecode packaged in individual `.class` files or `.jar` archives.	Executes Dalvik Executable packaged into a single consolidated `classes.dex` file.
Virtual Machine Architecture	**Stack-based architecture**. Pushes operands onto memory stack; requires more instruction cycles.	**Register-based architecture**. Stores operands in virtual registers; requires ~30% fewer instructions and executes faster on RISC/ARM mobile CPUs.
Constant Pool Sharing	Every single `.class` file has its own duplicate constant pool, wasting memory.	The `dx` tool merges and deduplicates constant string/symbol pools across all classes into one compact pool.
Memory & Footprint	High memory footprint; designed for desktop/servers.	Minimal RAM consumption; designed specifically for mobile devices with as little as 256MB RAM.
Process Model	Multiple applications often run within one shared JVM process.	Each Android app runs in an isolated Linux process with its own dedicated DVM instance.
Compilation Toolchain	`.java` $\to$ `javac` $\to$ `.class` $\to$ JVM.	`.java` $\to$ `javac` $\to$ `.class` $\to$ **`dx` tool** $\to$ `.dex` $\to$ DVM.

1.5 Android Development Tools & ADB Commands ⭐ Important

Android SDK Tool Ecosystem

Tool Name	Category	Core Purpose & Academic Details
Eclipse IDE	Integrated Dev Environment	The primary Java/XML integrated development environment specified in the syllabus.
ADT Plugin	Eclipse Extension	**Android Development Tools (ADT)** adds visual GUI layout editors, project wizards, XML autocompletion, and build automation directly into Eclipse.
Android SDK	API & Platform Libraries	Contains standard Android Java class libraries, API platform targets

Tool Name	Category	Core Purpose & Academic Details
		(API 16/17 Jelly Bean), documentation, and build tools.
AVD Manager	Device Emulator Manager	Allows developers to define and launch **Android Virtual Devices (AVD)** with custom screen sizes, API levels, SD cards, and RAM.
ADB (Android Debug Bridge)	Command-Line Client-Server Tool	Versatile CLI bridge that allows communication between PC terminal and connected emulators/USB devices.
DDMS (Dalvik Debug Monitor Server)	Monitoring & Profiling Suite	Integrated debugging tool for inspecting process thread allocation, heap dumps, LogCat streams, spoofing incoming SMS/calls, and taking screen captures.
dx Tool	DEX Bytecode Compiler	Converts compiled Java `.class` files into optimized Dalvik `.dex` files.
AAPT (Android Asset Packaging Tool)	Asset Resource Compiler	Compiles XML layouts, strings, and assets, and generates the resource pointer mapping class **`R.java`**.

Master ADB (Android Debug Bridge) Command Cheat Sheet (Top Exam Favorites):

ADB operates via a client-server architecture consisting of an ADB Client (running on PC), ADB Server (background process on PC), and ADB Daemon (adbd) running on the Android device.

```
# 1. List all connected physical devices and running emulators
adb devices

# 2. Install an APK file onto the target connected device
adb install MyApp.apk

# 3. Reinstall / Update existing application preserving user data
adb install -r MyApp.apk

# 4. Uninstall an application by its package name
adb uninstall com.example.helloworld

# 5. View real-time system diagnostic LogCat stream
adb logcat

# 6. Open a Linux command-line shell directly on the device
adb shell
```

```
# 7. Copy a file from PC to Android device storage
adb push C:\localfile.txt /sdcard/download/

# 8. Copy a file from Android device to PC
adb pull /sdcard/database.db C:\local_backup\

# 9. Forward host port to device port
adb forward tcp:8080 tcp:8080

# 10. Restart ADB background server
adb kill-server
adb start-server
```

📝 10-Mark Model Exam Answer: Explain the Android Software Architecture in detail with a neat layered diagram.

1. Introduction: Define Android as an open-source, Linux-based software stack composed of 5 distinct tiers.

2. Neat Diagram: Draw the 5-layer Android Stack Diagram showing Applications $\to$ Application Framework $\to$ C/C++ Libraries & Android Runtime $\to$ HAL $\to$ Linux Kernel.

3. Detailed Layer Explanation: Detail each layer with its core modules (Activity Manager, SurfaceFlinger, SQLite, Bionic, DVM, Binder IPC driver, LMK).

4. Role of DVM: Explain how DVM register-based architecture executes `.dex` files transformed by the `dx` tool.

5. Conclusion: Summarize how this modular stack provides device independence, robust process isolation, and optimized battery/RAM consumption.

📌 Quick Revision Sheet — Unit I

Android: Open-source, Linux-based mobile OS created by Andy Rubin / OHA / Google.

5 Stack Layers: Applications, App Framework, Libraries/Runtime, HAL, Linux Kernel.

Linux Kernel: Manages Binder IPC, Low Memory Killer (LMK), Wakelocks, and hardware drivers.

Dalvik VM: Register-based VM executing compact `.dex` files.

dx Tool: Merges multiple `.class` files into a single `classes.dex` with deduplicated constant pools.

ADB: Android Debug Bridge CLI tool (`adb devices`, `adb install`, `adb logcat`, `adb shell`).

🧠 TOPPER MEMORY TRICKS — UNIT I

- **5 Architecture Layers Mnemonic: A-F-L-R-H-K** $\to$ **A**ll **F**rogs **L**ike **R**unning **H**ome to **K**ernels (Applications, Framework, Libraries, Runtime, HAL, Kernel).
- **Framework Managers Mnemonic: A-W-C-V-N-R-L-T** $\to$ Activity, Window, Content Provider, View System, Notification, Resource, Location, Telephony.
- **Core ADB Commands: D-I-U-L-S-P** $\to$ **D**evices, **I**nstall, **U**ninstall, **L**ogcat, **S**hell, **P**ush/Pull.

II

Unit II: Object Oriented Programming Using Java & JVM Architecture

Java Foundation · Weightage: 20-25 Marks

> 📌 **Unit Syllabus Overview & Learning Objectives**

2.1 Core OOP Principles & Java Access Modifiers ⭐ Important

> **The 4 Pillars of Object-Oriented Programming (OOP)**
>
> - **Class:** A user-defined prototype or blueprint from which individual objects are created. Contains data fields (state) and member methods (behavior).
> - **Object:** A runtime concrete entity with a distinct identity, state, and behavior, instantiated in Heap memory using the `new` keyword.
> - **Encapsulation:** The technique of bundling data variables and the methods that manipulate them into a single unit (class), while restricting direct access using `private` access modifiers (Data Hiding via Getter and Setter methods).
> - **Abstraction:** Representing essential features of an entity without including background implementation details (achieved via *Abstract Classes* and *Interfaces*).

Java Access Modifiers Visibility Matrix:

Access Modifier	Within Same Class	Within Same Package	Subclass in Different Package	World (Everywhere)
`private`	✅ Yes	❌ No	❌ No	❌ No
`default` (Package-Private)	✅ Yes	✅ Yes	❌ No	❌ No
`protected`	✅ Yes	✅ Yes	✅ Yes (via inheritance)	❌ No
`public`	✅ Yes	✅ Yes	✅ Yes	✅ Yes

2.2 Inheritance & Polymorphism 🔥 Very Important

> **Inheritance Mechanics & Types**

Types of Inheritance in Java:

- **Single Inheritance:** Class B extends Class A ($A \to B$).
- **Multilevel Inheritance:** Class C extends Class B, which extends Class A ($A \to B \to C$).
- **Hierarchical Inheritance:** Multiple subclasses extend a single superclass ($A \to B$ and $A \to C$).
- **Multiple Inheritance (Prohibited in Java Classes):** Java prohibits a class from extending multiple classes to avoid the *"Diamond Ambiguity Problem"*. Multiple inheritance is achieved exclusively via **Interfaces**.

Polymorphism: Compile-Time vs Runtime

Method Overloading vs Method Overriding Master Comparison:

Feature	Method Overloading (Compile-Time)	Method Overriding (Runtime)
Definition	Multiple methods in the same class sharing the same name but having distinct parameter signatures.	Subclass provides a specific implementation of a method already declared in its superclass.
Location	Occurs within the **same class**.	Occurs between **superclass and subclass** (requires inheritance).
Method Signature	Must have **different parameters** (count, data types, or sequence).	Must have the **exact same signature** (name and parameters).
Return Type	Can be different or same.	Must be same (or covariant return type).
Binding Time	Static binding (resolved by compiler).	Dynamic binding (resolved by JVM at runtime).
Private/Static/Final Methods	Can overload private, static, and final methods.	Cannot override private, static, or final methods!

2.3 Interfaces vs Abstract Classes 🔥 Very Important

Feature	Abstract Class	Interface
Declaration Keyword	Declared using `abstract class`.	Declared using `interface` keyword.
Method Types	Can contain both **abstract methods** (without body) and **concrete methods** (with body).	Contains default 100% abstract methods (default/static methods allowed since Java 8).
Field Variables	Can declare static, non-static, final, and non-final instance variables.	Variables are implicitly `public static final` (constants only).
Multiple Inheritance	A subclass can extend only **one** abstract class (Single inheritance).	A class can implement **multiple** interfaces (`implements A, B, C`).
Constructor	Can declare constructors (called during subclass instantiation).	Cannot declare constructors (cannot be instantiated).
Performance Speed	Faster execution (direct method dispatch table).	Slightly slower due to dynamic interface lookup.

2.4 Multithreading & Thread Synchronization in Java & Android 🔥 Very Important

Multithreading Definition

📊 Visual Flow: Java Thread 5-State Life Cycle Diagram

Two Ways to Create Threads in Java:

9. **Extending `java.lang.Thread` Class:** Override `run()` method and invoke `start()`. (Limits further inheritance).
10. **Implementing `java.lang.Runnable` Interface (Preferred):** Implement `run()` method, pass instance to `Thread` object. Leaves class free to extend other classes.

Android Main UI Thread & ANR (Application Not Responding) Alert

2.5 JVM Architecture & Garbage Collection 🔥 10-Mark Guaranteed Question

Java Virtual Machine (JVM) Architecture

📊 Layered Diagram: Complete JVM Internal Architecture

The 5 Runtime Memory Areas in Detail:

11. **Method Area (Class Area):** Shared among all threads. Stores class-level data: per-class structures, runtime constant pool, static variables, field data, and constructor/method bytecode.
12. **Heap Area:** Shared memory pool where all object instances and arrays are dynamically allocated via `new`. Managed automatically by the Garbage Collector.
13. **JVM Stack:** Created per thread. Contains *Stack Frames* pushed upon method invocation and popped upon method return. Stores local variables, partial calculation results, and return values.
14. **Program Counter (PC) Registers:** Stores the memory address of the JVM instruction currently being executed by that thread.
15. **Native Method Stack:** Stores frame data for native C/C++ methods invoked via JNI.

2.6 Essential Exam Java Programs with Explanations 🔥 Practice All 7

Program 1: Class, Object & Parameterized Constructor

```
class Student {
    int rollNo;
    String name;
    String course;

    // Parameterized Constructor
    Student(int r, String n, String c) {
        rollNo = r;
        name = n;
        course = c;
    }

    void display() {
        System.out.println("Roll No: " + rollNo + " | Name: " + name + " | Course: " + course);
    }
```

```
    public static void main(String[] args) {
        Student s1 = new Student(101, "Aakash Rajawat", "BCA");
        Student s2 = new Student(102, "Rohan Sharma", "BCA");
        s1.display();
        s2.display();
    }
}
```

Program 2: Multilevel Inheritance with super Keyword

```
class Person {
    String name;
    Person(String name) { this.name = name; }
}

class Employee extends Person {
    double salary;
    Employee(String name, double salary) {
        super(name); // Calling superclass constructor
        this.salary = salary;
    }
}

class Manager extends Employee {
    String department;
    Manager(String name, double salary, String dept) {
        super(name, salary);
        this.department = dept;
    }

    void showDetails() {
        System.out.println("Manager Name: " + name);
        System.out.println("Salary: $" + salary);
        System.out.println("Department: " + department);
    }

    public static void main(String[] args) {
        Manager m = new Manager("Vikram Singh", 75000, "Android Dev");
        m.showDetails();
    }
}
```

Program 3: Method Overloading (Compile-Time Polymorphism)

```
class AreaCalculator {
    // Overloaded method for square
    double calculateArea(double side) {
        return side * side;
    }

    // Overloaded method for rectangle
    double calculateArea(double length, double breadth) {
        return length * breadth;
```

```
    }

    // Overloaded method for circle
    double calculateArea(float radius) {
        return 3.14159 * radius * radius;
    }

    public static void main(String[] args) {
        AreaCalculator calc = new AreaCalculator();
        System.out.println("Square Area (side=4): " + calc.calculateArea(4.0));
        System.out.println("Rectangle Area (4x6): " + calc.calculateArea(4.0, 6.0));
        System.out.println("Circle Area (r=3): " + calc.calculateArea(3.0f));
    }
}
```

Program 4: Method Overriding (Runtime Dynamic Binding)

```
class Bank {
    double getInterestRate() { return 0.0; }
}

class SBI extends Bank {
    @Override
    double getInterestRate() { return 6.75; }
}

class HDFC extends Bank {
    @Override
    double getInterestRate() { return 7.25; }
}

public class TestOverriding {
    public static void main(String[] args) {
        Bank b1 = new SBI();  // Upcasting (Dynamic Binding)
        Bank b2 = new HDFC();
        System.out.println("SBI Interest Rate: " + b1.getInterestRate() + "%");
        System.out.println("HDFC Interest Rate: " + b2.getInterestRate() + "%");
    }
}
```

Program 5: Interface Implementation (Multiple Inheritance via Interfaces)

```
interface Printable {
    void print();
}

interface Showable {
    void show();
}

class ReportDocument implements Printable, Showable {
    @Override
    public void print() {
```

```
        System.out.println("Document is sending to printer...");
    }

    @Override
    public void show() {
        System.out.println("Displaying Document on screen...");
    }

    public static void main(String[] args) {
        ReportDocument doc = new ReportDocument();
        doc.show();
        doc.print();
    }
}
```

Program 6: Abstract Class & Pure Abstraction

```
abstract class Vehicle {
    String brand;
    Vehicle(String brand) { this.brand = brand; }

    // Abstract method (no body)
    abstract void startEngine();

    // Concrete method
    void displayBrand() {
        System.out.println("Vehicle Brand: " + brand);
    }
}

class Car extends Vehicle {
    Car(String brand) { super(brand); }

    @Override
    void startEngine() {
        System.out.println("Starting Car engine with Push-Button!");
    }

    public static void main(String[] args) {
        Vehicle myCar = new Car("Toyota");
        myCar.displayBrand();
        myCar.startEngine();
    }
}
```

Program 7: Multithreading with Runnable & Synchronization

```
class BankAccount {
    private int balance = 1000;

    // Synchronized method prevents race conditions
    public synchronized void withdraw(int amount, String threadName) {
        if (balance >= amount) {
```

```
            System.out.println(threadName + " is withdrawing $" + amount);
            balance -= amount;
            System.out.println(threadName + " completed. Remaining Balance: $" + balance);
        } else {
            System.out.println(threadName + " failed. Insufficient funds!");
        }
    }
}

class CustomerTask implements Runnable {
    private BankAccount account;
    private String name;

    CustomerTask(BankAccount acc, String name) {
        this.account = acc;
        this.name = name;
    }

    @Override
    public void run() {
        account.withdraw(600, name);
    }

    public static void main(String[] args) {
        BankAccount sharedAcc = new BankAccount();
        Thread t1 = new Thread(new CustomerTask(sharedAcc, "User-1"));
        Thread t2 = new Thread(new CustomerTask(sharedAcc, "User-2"));

        t1.start();
        t2.start();
    }
}
```

📝 **10-Mark Model Exam Answer: Explain JVM Architecture with neat diagram, runtime memory areas, and execution engine components.**

1. Introduction: Explain JVM as the core virtual execution engine enabling Java platform independence ("Write Once, Run Anywhere").

2. Neat Diagram: Draw the 4-tier JVM Diagram: ClassLoader Subsystem $\to$ 5 Runtime Data Areas $\to$ Execution Engine (Interpreter, JIT, GC) $\to$ Native Interface.

3. Runtime Memory Breakdown: Explain all 5 areas (Method Area, Heap Area, JVM Stack, PC Registers, Native Method Stack) clearly distinguishing shared vs per-thread memory.

4. Execution Engine: Explain the role of Interpreter, JIT (Just-In-Time) compiler, and Garbage Collection.

5. Conclusion: Summarize how JVM decouples bytecode from physical CPU architecture.

📌 Quick Revision Sheet — Unit II

Encapsulation: Data hiding using private fields + public getter/setter methods.

Inheritance: Reusing code using `extends` keyword (Single, Multilevel, Hierarchical).

Overloading vs Overriding: Overloading = same class, different params; Overriding = subclass, same signature.

Interface: 100% abstract contract implemented via `implements`.

JVM Memory: Heap (Objects), Stack (Local variables), Method Area (Class code).

Thread States: New $\to$ Runnable $\to$ Running $\to$ Blocked $\to$ Dead.

🧠 TOPPER MEMORY TRICKS — UNIT II

- **JVM Memory Areas Mnemonic: M-H-S-P-N** $\to$ **M**ethod, **H**eap, **S**tack, **P**C Register, **N**ative Stack.
- **OOP 4 Pillars Mnemonic: A-P-I-E** $\to$ **A**bstraction, **P**olymorphism, **I**nheritance, **E**ncapsulation.

III

Unit III: Development Tools & Environment Setup

Tooling & Workflow · Weightage: 15-20 Marks

> 📌 **Unit Syllabus Overview & Learning Objectives**

3.1 Installing & Configuring Eclipse IDE with ADT Plugin 🔥 Very Important

> **Syllabus Environment Context vs. Modern Industry Standard**
>
> **University Exam Requirement:** The official BCA curriculum explicitly specifies **Eclipse IDE** with the **Android Development Tools (ADT) Plugin** as the canonical toolchain for exam questions.
>
> *Industry Note:* Google transitioned from Eclipse ADT to **Android Studio** (IntelliJ IDEA + Gradle) as the official IDE in 2015. For university theory exams, provide the Eclipse ADT steps as detailed below; for modern industry development, use Android Studio (Ladybug / Koala).

> 📊 **Visual Flowchart: End-to-End Android Development & Deployment Workflow**

Step-by-Step Installation & Configuration Procedure:

16. **Install Java Development Kit (JDK 6 or 7):** Set `JAVA_HOME` and add `bin` folder to the system `PATH` variable.
17. **Download & Extract Eclipse IDE:** Use Eclipse for Java Developers (Indigo, Juno, or Kepler).
18. **Install ADT (Android Development Tools) Plugin in Eclipse:**
    • Open Eclipse and navigate to `Help $\to$ Install New Software...`
    • Click **Add...** and enter Name: *ADT Plugin* and Location URL: `https://dl-ssl.google.com/android/eclipse/`
    • Select Developer Tools (DDMS, Android Development Tools, Hierarchy Viewer) and click **Finish**.
19. **Download Android SDK Manager:** Download SDK platforms (API 16/17 Jelly Bean), Android SDK Build-tools, Platform-tools (ADB, fastboot).
20. **Link SDK to Eclipse:** Open `Window $\to$ Preferences $\to$ Android` and browse to the extracted Android SDK root directory path.

3.2 Android Virtual Device (AVD / Emulator) ⭐ Important

> **Definition — AVD (Android Virtual Device)**

Key Configuration Parameters in AVD Manager:

- **AVD Name:** Unique identifier (e.g., `JellyBean_Nexus4`).
- **Device Definition:** Screen diagonal size and resolution (e.g., Nexus 4 - 768 x 1280 pixels, 4.7 inches).
- **Target API Level:** The Android platform target (e.g., *Android 4.2.2 - API Level 17*).
- **CPU/ABI Architecture:** `ARM (armeabi-v7a)` or `Intel Atom (x86)` (Intel Atom combined with Intel HAXM provides near-native execution speed).
- **Memory Options:** Device RAM allocation (e.g., `512 MB` or `768 MB`) and VM Heap size (e.g., `32 MB`).

- **SD Card Storage:** Virtual SD card image capacity (e.g., `100 MB` or `512 MB`) for testing external storage file reads/writes.

3.3 Android Project Directory Structure & AndroidManifest.xml 🔥 Very Important

Folder / File	Full Role & Academic Significance
`src/`	Contains all Java source code packages and classes (e.g., `com.example.myapp.MainActivity.java`).
`gen/R.java`	**Auto-generated resource dictionary file** created by the AAPT tool. Maps XML layouts, strings, and drawable assets to unique 32-bit hexadecimal integer IDs (e.g., `R.id.btnSubmit = 0x7f080001;`). **Rule: Never edit R.java manually!**
`res/layout/`	Contains XML layout definition files defining the visual UI structure (e.g., `activity_main.xml`).
`res/drawable-xx/`	Stores graphic raster images (PNG, JPG) and XML vector drawables partitioned by screen density (`drawable-ldpi`, `drawable-mdpi`, `drawable-hdpi`, `drawable-xhdpi`).
`res/values/`	Stores XML resource files: `strings.xml` (text strings), `colors.xml` (hex colors), `styles.xml` (themes), `dimens.xml` (margin/padding dimensions).
`res/menu/`	XML files defining action bar options menus and context menus.
`assets/`	Raw uncompiled asset files (custom TTF fonts, local HTML/JS web assets, raw audio clips) accessed via `AssetManager`.
`bin/`	Contains output build artifacts: compiled `classes.dex`, resources, and the generated packaged application file **`MyApp.apk`**.
`AndroidManifest.xml`	The **root configuration and declaration manifest** for the entire Android application.

Detailed Breakdown of AndroidManifest.xml Tags:

```
<?xml version="1.0" encoding="utf-8"?>
<!-- 1. Root Manifest Tag with Package Definition -->
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.helloworld"
    android:versionCode="1"
    android:versionName="1.0" >
```

```
    <!-- 2. Minimum and Target SDK API Level declaration -->
    <uses-sdk
        android:minSdkVersion="14"
        android:targetSdkVersion="17" />

    <!-- 3. System Hardware/Security Permissions -->
    <uses-permission android:name="android.permission.INTERNET" />
    <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />

    <!-- 4. Application Configuration Block -->
    <application
        android:allowBackup="true"
        android:icon="@drawable/ic_launcher"
        android:label="@string/app_name"
        android:theme="@style/AppTheme" >

        <!-- 5. Activity Registration -->
        <activity
            android:name=".MainActivity"
            android:label="@string/app_name" >

            <!-- 6. Intent Filter designating Launcher Entry Point -->
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

        <!-- Additional Activities must also be registered here -->
        <activity android:name=".SecondActivity" />

    </application>
</manifest>
```

3.4 Creating "Hello World" Code Example 🔥 Very Important

A standard Android app requires at least 3 synchronized files:

1. XML Layout (res/layout/activity_main.xml):

```
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="16dp" >

    <TextView
        android:id="@+id/txtGreeting"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_centerInParent="true"
        android:text="@string/hello_message"
        android:textColor="#065f46"
        android:textSize="22sp"
```

```
        android:textStyle="bold" />

</RelativeLayout>
```

2. String Resource (res/values/strings.xml):

```
<resources>
    <string name="app_name">HelloWorldApp</string>
    <string name="hello_message">Hello World from Android Jelly Bean!</string>
</resources>
```

3. Java Activity (src/com/example/helloworld/MainActivity.java):

```
package com.example.helloworld;

import android.app.Activity;
import android.os.Bundle;
import android.widget.TextView;

public class MainActivity extends Activity {

    // First lifecycle method invoked when activity starts
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);

        // Inflates and renders the XML UI layout onto the device window
        setContentView(R.layout.activity_main);

        // Finding view reference from R.java ID pointer
        TextView txt = (TextView) findViewById(R.id.txtGreeting);
    }
}
```

3.5 Deploying Application on Physical USB-Connected Android Device 🔥 5-Mark Exam Question

> **USB Deployment Overview**

Step-by-Step USB Deployment Procedure:

21. **Enable Developer Options on Device:**
    - On Android 4.2+ (Jelly Bean), open phone `Settings $\to$ About Phone`.
    - Tap **"Build Number"** 7 times continuously until the prompt *"You are now a developer!"* appears.

22. **Enable USB Debugging:**
    - Go to `Settings $\to$ Developer Options`.
    - Toggle on **"USB Debugging"** and accept the security confirmation prompt.

23. **Install OEM USB Driver on PC:**
    - On Windows, install the specific USB driver for the device manufacturer (Samsung, Google Nexus, Motorola).

24. **Connect Hardware Device via USB Cable:**
    • Connect device to PC via USB. When the phone prompts *"Allow USB Debugging?"*, check *"Always allow from this computer"* and tap **OK**.

25. **Verify Connection in Eclipse / ADB:**
    • Open command terminal on PC and execute `adb devices`. The phone serial number should be listed as `device`.
    • Alternatively, open **DDMS View** in Eclipse; the phone should appear in the Devices list.

26. **Run Application from Eclipse:**
    • In Eclipse, right-click the Android Project $\to$ `Run As $\to$ Android Application`.
    • In the Android Device Chooser dialog, select the connected physical phone and click **OK**. The APK will be transferred, installed, and launched automatically!

> 📝 **10-Mark Model Exam Answer: Describe the complete Android application development environment setup using Eclipse with ADT and explain the project directory structure.**
>
> **1. Introduction:** Detail the role of Eclipse IDE, ADT Plugin, and Android SDK in the development lifecycle.
>
> **2. Setup Steps:** Detail JDK installation $\to$ Eclipse configuration $\to$ ADT plugin installation $\to$ SDK linking.
>
> **3. Project Directory Tree:** Draw directory tree detailing `src/`, `gen/R.java`, `res/layout`, `res/values`, `res/drawable`, `bin/`, and `AndroidManifest.xml`.
>
> **4. Code Walkthrough:** Provide the Hello World XML Layout and Java Activity code.
>
> **5. USB Deployment:** List the 6-step procedure for enabling USB Debugging and deploying onto real hardware.

📌 Quick Revision Sheet — Unit III

Eclipse + ADT: Primary official IDE setup for syllabus development.
gen/R.java: Auto-generated file containing 32-bit hex resource IDs; NEVER EDIT.
AndroidManifest.xml: Core registration file for activities, permissions, and SDK versions.
AVD: Android Virtual Device emulator configuration for software testing.
USB Debugging: Developer setting required to deploy and debug apps on real phones.
setContentView(): Method that attaches XML layout to Java Activity.

🧠 TOPPER MEMORY TRICKS — UNIT III

- **Project Tree Mnemonic: S-G-R-B-A-M** $\to$ **S**rc, **G**en (R.java), **R**es, **B**in, **A**ssets, **M**anifest.
- **USB Deployment Mnemonic: B-D-C-V-R** $\to$ **B**uild number (7 taps), **D**ebugging on, **C**able connect, **V**erify ADB, **R**un app.

IV

Unit IV: User Interface Architecture, Activity Life Cycle & UI Design

UI & Lifecycle · Weightage: 25-30 Marks

📌 Unit Syllabus Overview & Learning Objectives

4.1 Application Context & Intent Passing 🔥 Very Important

Application Context Definition

Intent Definition & Types

📊 Visual Diagram: Explicit vs Implicit Intent Passing Mechanism

Explicit vs Implicit Intents Code Implementation:

```
// ========================================
// 1. EXPLICIT INTENT (Navigating Inside App)
// ========================================
Intent explicitIntent = new Intent(MainActivity.this, SecondActivity.class);
// Passing data payloads via Key-Value Extras
explicitIntent.putExtra("STUDENT_NAME", "Aakash Rajawat");
explicitIntent.putExtra("STUDENT_ID", 101);
startActivity(explicitIntent);

// In SecondActivity.java: Receiving Data
Intent receivedIntent = getIntent();
String name = receivedIntent.getStringExtra("STUDENT_NAME");
int id = receivedIntent.getIntExtra("STUDENT_ID", 0);



// ========================================
// 2. IMPLICIT INTENTS (Invoking System Apps)
// ========================================
// Example A: Open Web URL in default device browser
Intent openWeb = new Intent(Intent.ACTION_VIEW, Uri.parse("https://www.google.com"));
startActivity(openWeb);

// Example B: Open Phone Dialer with prefilled number
Intent dialPhone = new Intent(Intent.ACTION_DIAL, Uri.parse("tel:9876543210"));
startActivity(dialPhone);

// Example C: Share text content via Messaging / Social apps
Intent shareIntent = new Intent(Intent.ACTION_SEND);
shareIntent.setType("text/plain");
shareIntent.putExtra(Intent.EXTRA_TEXT, "Checkout my Android Topper Notes!");
startActivity(Intent.createChooser(shareIntent, "Share notes via..."));
```

4.2 Activity Life Cycle (The 7 Callback States & Transitions) 🔥 Guaranteed 10-Mark Exam Driver

> **Activity Life Cycle Definition**

> 📊 **Visual State Flowchart: Android Activity Complete 7-Stage Life Cycle**

The 7 Lifecycle Callbacks Explained in Depth:

Callback Method	When Triggered	Developer Action & Best Practice
`onCreate(Bundle)`	Called once when activity instance is first initialized in memory.	Mandatory setup: Call `setContentView()`, bind view variables using `findViewById()`, and initialize database connections.
`onStart()`	Called when activity becomes visually visible on screen.	Register broadcast receivers or UI listeners.
`onResume()`	Called when activity enters the foreground and starts receiving user touch/key events.	Start animations, acquire camera, initialize GPS location updates, resume media playback.
`onPause()`	Called when another activity receives partial focus (e.g., incoming phone call banner, translucent dialog, screen lock).	Commit unsaved draft edits, pause video/audio playback, release heavy system resources (GPS/Camera) to conserve battery.
`onStop()`	Called when activity is 100% hidden and obscured by another full-screen activity.	Save persistent data to SQLite database / SharedPreferences, stop heavy background animations.
`onRestart()`	Called after `onStop()` when user navigates back to the existing stopped activity.	Special refresh logic before `onStart()` is executed again.
`onDestroy()`	Called right before the activity is terminated by calling `finish()` or killed by OS for memory reclaim.	Release all remaining thread references, close database helpers, prevent memory leaks.

Saving State during Screen Rotation (Configuration Changes):

> **Handling Configuration Changes**

```
// 1. Save transient state before activity destruction
@Override
protected void onSaveInstanceState(Bundle outState) {
    super.onSaveInstanceState(outState);
    outState.putString("SAVED_TEXT", editInput.getText().toString());
    outState.putInt("USER_SCORE", score);
}
```

```
// 2. Restore state when activity is recreated
@Override
protected void onRestoreInstanceState(Bundle savedInstanceState) {
    super.onRestoreInstanceState(savedInstanceState);
    String savedText = savedInstanceState.getString("SAVED_TEXT");
    int savedScore = savedInstanceState.getInt("USER_SCORE");
    editInput.setText(savedText);
}
```

4.3 Screen Sizes, Densities & dp/sp Mathematical Formulations ⭐ Important

Screen Density Categories & Baseline

Density Bucket	DPI Range	Scaling Factor	100 dp in Pixels (px)	Drawable Folder
`ldpi` (Low)	~120 dpi	0.75x	75 px	`res/drawable-ldpi/`
`mdpi` (Medium - Baseline)	~160 dpi	1.0x (1:1)	100 px	`res/drawable-mdpi/`
`hdpi` (High)	~240 dpi	1.5x	150 px	`res/drawable-hdpi/`
`xhdpi` (Extra-High)	~320 dpi	2.0x	200 px	`res/drawable-xhdpi/`
`xxhdpi` (Extra-Extra-High)	~480 dpi	3.0x	300 px	`res/drawable-xxhdpi/`

🧱 Mathematical Conversion Formula

dp vs sp vs px Master Comparison:

- **dp (Density-Independent Pixels):** Virtual pixel unit that scales proportionally with screen density. Used for **layout width, height, margins, and padding**.
- **sp (Scale-Independent Pixels):** Virtual pixel unit that scales with screen density *AND* user-defined accessibility font-size preferences. Used exclusively for **typography text sizes (`android:textSize="18sp"`)**.
- **px (Physical Hardware Pixels):** Raw hardware screen pixels. **Avoid using px in Android** because a 100px button looks large on a low-resolution phone but shrinks to microscopic size on a high-density 4K display!

4.4 Android Layout Managers Deep Dive 🔥 Very Important

View vs ViewGroup Hierarchy

Layout Manager	Arrangement Logic	Key XML Attributes	Best Practical Use Case
LinearLayout	Arranges child views in a single linear row (`horizontal`) or column (`vertical`).	`android:orientation="vertical|horizontal"android:layout_weight="1"`	Simple vertical input forms, toolbars, uniform button rows.
RelativeLayout	Positions child views relative to parent boundaries or relative to sibling views by ID.	`android:layout_below="@id/txtTitle"android:layout_toRightOf="@id/imgIcon"android:layout_centerInParent="true"`	Complex flat UI layouts without nesting overhead.
TableLayout	Arranges child views into regular grid rows and columns using `<TableRow>` elements.	`android:stretchColumns="*"android:layout_column="1"`	Spreadsheet grids, calculator keypads, tabular summary cards.
FrameLayout	Designed to	`android:layout_gravity="center|bottom"`	Fragment

Layout Manager	Arrangement Logic	Key XML Attributes	Best Practical Use Case
	hold a single child view or stack overlapping views (Z-axis stack).		containers, cards with badges or floating action icons.
ScrollView	Vertical scroll container that holds a single child layout for handling screen overflow.	`<ScrollView><LinearLayout ...>...</LinearLayout></ScrollView>`	Long forms or scrollable text articles.

XML Code Example — Combining LinearLayout & Weight:

```
<!-- LinearLayout distributing width 50%-50% using layout_weight -->
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:weightSum="2" >

    <Button
        android:id="@+id/btnCancel"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:text="Cancel" />

    <Button
        android:id="@+id/btnSave"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
```

```
        android:layout_weight="1"
        android:text="Save" />
</LinearLayout>
```

4.5 UI Form Controls, Widgets & Event Listeners 🔥 Very Important

Widget Name	Visual Role	Key Java Listener Interface
`TextView`	Displays non-editable text labels on screen.	`txt.setText("Hello");`
`EditText`	Editable input field (supports inputType: password, email, phone).	`edit.getText().toString();`
`Button` / `ImageButton`	Standard clickable button triggering an immediate action.	`View.OnClickListener`
`ToggleButton`	Displays a two-state button with an ON/OFF light indicator.	`CompoundButton.OnCheckedChangeListener`
`CheckBox`	Allows users to select multiple options from a set.	`chk.isChecked();`
`RadioButton` & `RadioGroup`	Exclusive single-choice selection widget within a group.	`RadioGroup.OnCheckedChangeListener`
`Spinner`	Dropdown selector displaying a list of options.	`AdapterView.OnItemSelectedListener`
`ListView`	Vertical scrollable list displaying data rows using an Adapter.	`AdapterView.OnItemClickListener`

Java Event Listener Binding Master Code:

```
// 1. Binding Button with OnClickListener
Button btnSubmit = (Button) findViewById(R.id.btnSubmit);
btnSubmit.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        Toast.makeText(MainActivity.this, "Submit Button Clicked!", Toast.LENGTH_SHORT).show();
    }
});

// 2. RadioGroup Single Selection
RadioGroup radioGroupGender = (RadioGroup) findViewById(R.id.radioGroupGender);
radioGroupGender.setOnCheckedChangeListener(new RadioGroup.OnCheckedChangeListener() {
    @Override
    public void onCheckedChanged(RadioGroup group, int checkedId) {
        RadioButton selectedRadio = (RadioButton) findViewById(checkedId);
        String gender = selectedRadio.getText().toString();
```

```
    }
});

// 3. Populating Spinner with ArrayAdapter
Spinner spinnerCourse = (Spinner) findViewById(R.id.spinnerCourse);
String[] courses = { "BCA", "MCA", "B.Tech", "M.Tech", "B.Sc IT" };
ArrayAdapter<String> adapter = new ArrayAdapter<String>(this,
    android.R.layout.simple_spinner_item, courses);
adapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);
spinnerCourse.setAdapter(adapter);

spinnerCourse.setOnItemSelectedListener(new AdapterView.OnItemSelectedListener() {
    @Override
    public void onItemSelected(AdapterView<?> parent, View view, int position, long id) {
        String selectedCourse = courses[position];
    }
    @Override
    public void onNothingSelected(AdapterView<?> parent) {}
});
```

4.6 Menus, Dialogs & Toast Notifications ⭐ Important

1. Options Menu in Activity:

```
// Inflate Options Menu from XML (res/menu/main_menu.xml)
@Override
public boolean onCreateOptionsMenu(Menu menu) {
    getMenuInflater().inflate(R.menu.main_menu, menu);
    return true;
}

// Handle Action Bar / Menu item selection
@Override
public boolean onOptionsItemSelected(MenuItem item) {
    int id = item.getItemId();
    if (id == R.id.menu_settings) {
        Toast.makeText(this, "Settings Selected", Toast.LENGTH_SHORT).show();
        return true;
    } else if (id == R.id.menu_about) {
        Toast.makeText(this, "About App Selected", Toast.LENGTH_SHORT).show();
        return true;
    }
    return super.onOptionsItemSelected(item);
}
```

2. AlertDialog.Builder Code:

```
AlertDialog.Builder builder = new AlertDialog.Builder(MainActivity.this);
builder.setTitle("Confirm Exit");
builder.setMessage("Are you sure you want to close the application?");
builder.setIcon(android.R.drawable.ic_dialog_alert);

// Positive Button
```

```
builder.setPositiveButton("Yes", new DialogInterface.OnClickListener() {
    @Override
    public void onClick(DialogInterface dialog, int which) {
        finish(); // Terminate Activity
    }
});

// Negative Button
builder.setNegativeButton("No", new DialogInterface.OnClickListener() {
    @Override
    public void onClick(DialogInterface dialog, int which) {
        dialog.dismiss(); // Dismiss dialog
    }
});

AlertDialog alert = builder.create();
alert.show();
```

📝 **10-Mark Model Exam Answer: Explain the complete Activity Life Cycle with a neat state diagram and detail all 7 callback methods with practical scenarios.**

1. Introduction: Define Activity as the primary UI window and explain why the OS manages lifecycle states.

2. Neat Diagram: Draw the 7-stage Activity Lifecycle Diagram (`onCreate $\to$ onStart $\to$ onResume $\to$ Active Running $\to$ onPause $\to$ onStop $\to$ onDestroy` + `onRestart` loop).

3. Detailed Callback Analysis: Detail all 7 callbacks with their precise execution triggers and developer responsibilities.

4. Real-World Scenarios: Walk through incoming phone call scenario and Home button backgrounding.

5. Configuration Handling: Explain `onSaveInstanceState()` and `onRestoreInstanceState()`.

📌 Quick Revision Sheet — Unit IV

Explicit Intent: Targets specific class inside same application.

Implicit Intent: Requests action from system intent resolver (e.g., ACTION_VIEW).

7 Activity Callbacks: onCreate, onStart, onResume, onPause, onStop, onRestart, onDestroy.

dp vs sp: Use dp for layout dimensions, sp for text typography.

LinearLayout: Arranges views linearly; RelativeLayout positions views relative to siblings.

Toast: Non-intrusive short popup message.

🧠 TOPPER MEMORY TRICKS — UNIT IV

- **Activity 7 Callbacks Mnemonic: C-S-R-P-S-R-D** $\to$ **C**an **S**tudents **R**ead **P**roperly **S**o **R**esults **D**evelop (onCreate, onStart, onResume, onPause, onStop, onRestart, onDestroy).
- **Layout Managers Mnemonic: L-R-T-F-S** $\to$ **L**inear, **R**elative, **T**able, **F**rame, **S**croll.

V

Unit V: Database Management using SQLite

Database & Persistence · Weightage: 20-25 Marks

> 📌 **Unit Syllabus Overview & Learning Objectives**

5.1 Understanding SQLite & Supported Data Types 🔥 Very Important

> **Definition — SQLite in Android**

Key Architectural Features of SQLite:

- **Serverless & Embedded:** No database server process to start, configure, or administer. The database engine runs in-process with the application.
- **Zero Configuration:** No setup, initial installation, or administrator permissions required.
- **Single Disk File:** An entire database with multiple tables, indices, triggers, and views is contained within a single standalone file on disk.
- **ACID Transactional Compliance:** Supports Atomic, Consistent, Isolated, and Durable transactions (`beginTransaction()`, `setTransactionSuccessful()`, `endTransaction()`).
- **Dynamically Typed (Type Affinity):** Data types are associated with values themselves rather than strictly with columns.

Supported SQLite Storage Classes (Data Types):

Storage Class	Description	Java Mapping Type
`NULL`	Represents a null or undefined value.	`null`
`INTEGER`	Signed integer stored in 1, 2, 3, 4, 6, or 8 bytes depending on value magnitude.	`int`, `long`, `short`, `byte`
`REAL`	Floating point value stored as an 8-byte IEEE floating point number.	`float`, `double`
`TEXT`	Text string stored using database encoding (UTF-8, UTF-16BE, or UTF-16LE).	`String`
`BLOB`	Binary Large Object stored exactly as it was input (images, audio clips, serialized files).	`byte[]`

5.2 Connecting with Database & SQLiteOpenHelper 🔥 Very Important

SQLiteOpenHelper Class Mechanics

The 2 Essential Lifecycle Callbacks to Override:

27. `public void onCreate(SQLiteDatabase db)`:
    - Called automatically **only once** when the database file is first created on device storage.
    - Place all `CREATE TABLE` and initial seed data SQL statements inside this method.

28. `public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion)`:
    - Called automatically when the database `DATABASE_VERSION` number passed to the constructor is incremented.
    - Used to execute schema migration SQL (e.g., `ALTER TABLE` or `DROP TABLE IF EXISTS` followed by `onCreate(db)`).

getWritableDatabase() vs getReadableDatabase():

- **`getWritableDatabase()`:** Opens/creates the database in read-write mode. Required when performing `INSERT`, `UPDATE`, or `DELETE` operations.
- **`getReadableDatabase()`:** Opens the database in read-only mode if disk storage is full; otherwise opens in read-write mode. Used for `SELECT` queries.

5.3 Complete Student CRUD Database Java Application 🔥 10-Mark Guaranteed Question

Here is the complete, robust, exam-ready 2-file CRUD database implementation:

File 1: DatabaseHelper.java (Database OpenHelper & Operations):

```
package com.example.studentdb;

import android.content.ContentValues;
import android.content.Context;
import android.database.Cursor;
import android.database.sqlite.SQLiteDatabase;
import android.database.sqlite.SQLiteOpenHelper;

public class DatabaseHelper extends SQLiteOpenHelper {

    // Database Constants
    public static final String DATABASE_NAME = "StudentRegistry.db";
    public static final int DATABASE_VERSION = 1;
    public static final String TABLE_NAME = "students";

    // Column Names
    public static final String COL_ID = "ID";
    public static final String COL_NAME = "NAME";
    public static final String COL_COURSE = "COURSE";
    public static final String COL_MARKS = "MARKS";

    // Constructor
    public DatabaseHelper(Context context) {
        super(context, DATABASE_NAME, null, DATABASE_VERSION);
    }
```

```
// 1. Table Creation
@Override
public void onCreate(SQLiteDatabase db) {
    String CREATE_STUDENTS_TABLE = "CREATE TABLE " + TABLE_NAME + " (" +
            COL_ID + " INTEGER PRIMARY KEY, " +
            COL_NAME + " TEXT, " +
            COL_COURSE + " TEXT, " +
            COL_MARKS + " REAL)";
    db.execSQL(CREATE_STUDENTS_TABLE);
}

// 2. Table Upgrade
@Override
public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
    db.execSQL("DROP TABLE IF EXISTS " + TABLE_NAME);
    onCreate(db);
}

// ====================================================
// CRUD OPERATIONS (Create, Read, Update, Delete)
// ====================================================

// C - INSERT / CREATE Record
public boolean insertStudent(int id, String name, String course, double marks) {
    SQLiteDatabase db = this.getWritableDatabase();
    ContentValues contentValues = new ContentValues();
    contentValues.put(COL_ID, id);
    contentValues.put(COL_NAME, name);
    contentValues.put(COL_COURSE, course);
    contentValues.put(COL_MARKS, marks);

    long result = db.insert(TABLE_NAME, null, contentValues);
    return result != -1; // returns true if insert succeeded
}

// R - READ / VIEW ALL Records
public Cursor getAllStudents() {
    SQLiteDatabase db = this.getReadableDatabase();
    // Returns a Cursor pointer to query result set
    return db.rawQuery("SELECT * FROM " + TABLE_NAME, null);
}

// U - UPDATE Record by ID
public boolean updateStudent(int id, String name, String course, double marks) {
    SQLiteDatabase db = this.getWritableDatabase();
    ContentValues contentValues = new ContentValues();
    contentValues.put(COL_NAME, name);
    contentValues.put(COL_COURSE, course);
    contentValues.put(COL_MARKS, marks);

    int rowsAffected = db.update(TABLE_NAME, contentValues,
        COL_ID + " = ?", new String[]{ String.valueOf(id) });
```

```
        return rowsAffected > 0;
    }

    // D - DELETE Record by ID
    public boolean deleteStudent(int id) {
        SQLiteDatabase db = this.getWritableDatabase();
        int rowsDeleted = db.delete(TABLE_NAME,
            COL_ID + " = ?", new String[]{ String.valueOf(id) });
        return rowsDeleted > 0;
    }
}
```

File 2: MainActivity.java (UI Integration & Event Handling):

```
package com.example.studentdb;

import android.app.Activity;
import android.app.AlertDialog;
import android.database.Cursor;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;

public class MainActivity extends Activity {

    DatabaseHelper myDb;
    EditText editId, editName, editCourse, editMarks;
    Button btnAdd, btnViewAll, btnUpdate, btnDelete;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        // Initialize Database Helper
        myDb = new DatabaseHelper(this);

        // Bind UI Components
        editId = (EditText) findViewById(R.id.editId);
        editName = (EditText) findViewById(R.id.editName);
        editCourse = (EditText) findViewById(R.id.editCourse);
        editMarks = (EditText) findViewById(R.id.editMarks);

        btnAdd = (Button) findViewById(R.id.btnAdd);
        btnViewAll = (Button) findViewById(R.id.btnViewAll);
        btnUpdate = (Button) findViewById(R.id.btnUpdate);
        btnDelete = (Button) findViewById(R.id.btnDelete);

        // 1. ADD DATA ACTION
        btnAdd.setOnClickListener(new View.OnClickListener() {
            @Override
```

```
            public void onClick(View v) {
                boolean isInserted = myDb.insertStudent(
                    Integer.parseInt(editId.getText().toString()),
                    editName.getText().toString(),
                    editCourse.getText().toString(),
                    Double.parseDouble(editMarks.getText().toString())
                );
                if (isInserted)
                    Toast.makeText(MainActivity.this, "Student Added Successfully!",
Toast.LENGTH_SHORT).show();
                else
                    Toast.makeText(MainActivity.this, "Insertion Failed!",
Toast.LENGTH_SHORT).show();
            }
        });

        // 2. VIEW ALL DATA ACTION
        btnViewAll.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                Cursor res = myDb.getAllStudents();
                if (res.getCount() == 0) {
                    showMessage("Error", "No student records found in database.");
                    return;
                }

                // Iterate through Cursor pointer rows
                StringBuffer buffer = new StringBuffer();
                while (res.moveToNext()) {
                    buffer.append("ID: " + res.getString(0) + "\n");
                    buffer.append("Name: " + res.getString(1) + "\n");
                    buffer.append("Course: " + res.getString(2) + "\n");
                    buffer.append("Marks: " + res.getString(3) + "\n\n");
                }
                res.close(); // Always close cursor to prevent memory leaks

                // Display in AlertDialog popup modal
                showMessage("Student Records", buffer.toString());
            }
        });

        // 3. UPDATE DATA ACTION
        btnUpdate.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                boolean isUpdate = myDb.updateStudent(
                    Integer.parseInt(editId.getText().toString()),
                    editName.getText().toString(),
                    editCourse.getText().toString(),
                    Double.parseDouble(editMarks.getText().toString())
                );
                if (isUpdate)
                    Toast.makeText(MainActivity.this, "Record Updated Successfully!",
Toast.LENGTH_SHORT).show();
```

```
                else
                    Toast.makeText(MainActivity.this, "Update Failed!",
Toast.LENGTH_SHORT).show();
            }
        });

        // 4. DELETE DATA ACTION
        btnDelete.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                boolean isDeleted = myDb.deleteStudent(
                    Integer.parseInt(editId.getText().toString())
                );
                if (isDeleted)
                    Toast.makeText(MainActivity.this, "Record Deleted Successfully!",
Toast.LENGTH_SHORT).show();
                else
                    Toast.makeText(MainActivity.this, "Delete Failed!",
Toast.LENGTH_SHORT).show();
            }
        });
    }

    // Helper method to display AlertDialog
    public void showMessage(String title, String message) {
        AlertDialog.Builder builder = new AlertDialog.Builder(this);
        builder.setCancelable(true);
        builder.setTitle(title);
        builder.setMessage(message);
        builder.show();
    }
}
```

5.4 Android App to Database Architecture Flow ⭐ Important

📊 Layered Diagram: Complete Android App to SQLite Database Execution Flow

📝 10-Mark Model Exam Answer: Explain SQLite database management in Android with SQLiteOpenHelper class and write Java code for database creation, insertion, updating, and querying.

1. Introduction: Explain SQLite as an embedded serverless relational database engine.

2. SQLite Features: List 5 features: Serverless, zero-configuration, single-file disk persistence, ACID compliant, lightweight.

3. SQLiteOpenHelper: Detail `onCreate()`, `onUpgrade()`, `getWritableDatabase()`, and `getReadableDatabase()`.

4. Java Code: Write complete `DatabaseHelper.java` class with `insertStudent()`, `getAllStudents()`, `updateStudent()`, and `deleteStudent()`.

5. App-to-DB Flow: Draw the 4-stage architecture diagram showing Activity $\to$ Helper $\to$ SQLite Engine $\to$ Disk File.

📌 Quick Revision Sheet — Unit V

SQLite: Embedded, serverless, zero-config relational database.

SQLiteOpenHelper: Manages DB creation (`onCreate`) & upgrades (`onUpgrade`).

ContentValues: Key-Value container object used for insert & update.

Cursor: Pointer interface returned by queries to iterate result rows.

getWritableDatabase(): Opens DB in read/write mode for DML operations.

DB File Path: `/data/data/<pkg>/databases/<dbname>.db`.

🧠 TOPPER MEMORY TRICKS — UNIT V

- **SQLite 5 Storage Classes Mnemonic: N-I-R-T-B** $\to$ **N**ULL, **I**NTEGER, **R**EAL, **T**EXT, **B**LOB.
- **CRUD Operations: C-R-U-D** $\to$ **C**reate (Insert), **R**ead (Select), **U**pdate, **D**elete.

⚖️ 12 Master Comparison Tables
Quick Exam Revisions

1. Android OS vs Traditional Desktop OS (Windows / Linux)

Feature	Android Mobile OS	Traditional Desktop OS (Windows/Linux)
Target Hardware	Battery-powered touchscreen mobile devices (phones, tablets).	AC-powered desktop computers, laptops, servers.
Execution Engine	Dalvik / ART Virtual Machine executing `.dex` files.	Native CPU machine code / JVM executing `.class` files.
Power Management	Aggressive mobile power optimization via Wakelocks and sleep states.	Standard continuous AC power management.
Process Sandbox	Every app runs as a distinct Linux UID in an isolated sandbox.	Applications share user privileges within the user account.

2. JVM vs DVM (Dalvik Virtual Machine) vs ART

Feature	JVM (Standard Java)	DVM (Dalvik VM)	ART (Android Runtime)
Executable Format	Java Bytecode (`.class`)	Dalvik Executable (`.dex`)	Ahead-Of-Time Native Binary (`.oat`)
Architecture	Stack-Based	Register-Based	Register-Based
Compilation Type	JIT (Just-In-Time)	JIT (Just-In-Time since Android 2.2)	AOT (Ahead-Of-Time at install) + Profile-guided JIT
RAM Footprint	High	Very Low (Optimized for 256MB+)	Moderate (Faster app startup)

3. Method Overloading vs Method Overriding

Feature	Method Overloading	Method Overriding
Scope	Occurs within the **same class**.	Occurs between **superclass and subclass** (Inheritance).
Method Signature	Must have **different parameters**.	Must have the **exact same signature**.
Binding	Compile-time (Static binding).	Runtime (Dynamic method dispatch).

Feature	Method Overloading	Method Overriding
Return Type	Can be same or different.	Must be identical (or covariant).

4. Abstract Class vs Interface

Feature	Abstract Class	Interface
Keyword	`abstract class`	`interface`
Method Types	Abstract and Concrete methods.	100% Abstract methods (Default methods in Java 8+).
Multiple Inheritance	Single inheritance only (`extends`).	Multiple inheritance supported (`implements A, B`).
Variables	Instance, static, final, non-final fields.	Implicitly `public static final` (Constants).

5. Explicit Intent vs Implicit Intent

Feature	Explicit Intent	Implicit Intent
Target Component	Explicitly designates the exact target Java class.	Designates an abstract action (e.g., `ACTION_VIEW`).
Resolution	Direct execution within the same app.	Resolved by Android OS matching `<intent-filter>` tags.
Use Case	Navigating between screens inside your app.	Delegating tasks to external apps (Browser, Camera, Dialer).

6. LinearLayout vs RelativeLayout vs TableLayout vs FrameLayout

Layout Manager	Arrangement Mechanism	Key Attribute
`LinearLayout`	Linear sequence (Horizontal or Vertical).	`android:orientation`, `android:layout_weight`
`RelativeLayout`	Positions relative to parent bounds or sibling view IDs.	`layout_below`, `layout_toRightOf`
`TableLayout`	Grid matrix of rows and columns.	`<TableRow>`, `android:stretchColumns`
`FrameLayout`	Stacks overlapping views along Z-axis.	`android:layout_gravity`

7. dp vs sp vs px Units

Unit	Stands For	Scaling Behavior	Best Practice Recommendation
`dp` / `dip`	Density-Independent Pixels	Scales with physical screen density.	Use for **view widths, heights, margins, and padding**.
`sp`	Scale-Independent Pixels	Scales with screen density + user font accessibility settings.	Use exclusively for **font sizes (`textSize`)**.
`px`	Physical Hardware Pixels	No scaling; 1:1 hardware pixel mapping.	**Avoid using px** in Android layouts.

8. Button vs ToggleButton vs CheckBox vs RadioButton

UI Control	State Behavior	Selection Multiplicity	Visual Display
`Button`	Momentary Stateless Press	Single Action Trigger	Rectangular Clickable Push Button
`ToggleButton`	Dual State (ON / OFF)	Single Toggle	Button with Light Indicator Bar
`CheckBox`	Dual State (Checked / Unchecked)	Multiple Independent Selections	Square Box with Checkmark
`RadioButton`	Mutually Exclusive State	Single Selection inside RadioGroup	Circular Selection Dot

9. getWritableDatabase() vs getReadableDatabase()

Feature	getWritableDatabase()	getReadableDatabase()
Access Mode	Read-Write mode.	Read-Only mode if storage is full; else Read-Write.
Target Operations	`INSERT`, `UPDATE`, `DELETE`, `CREATE TABLE`.	`SELECT` queries, reporting, searches.

10. execSQL() vs insert() / update() / delete() Convenience Methods

Feature	execSQL(String sql)	insert() / update() / delete() Methods
SQL Statement	Raw SQL string executed directly.	Constructed safely using `ContentValues` parameters.
Return Value	`void` (no feedback on affected	Returns `long` row ID or `int` count

Feature	execSQL(String sql)	insert() / update() / delete() Methods
	rows).	of affected rows.
SQL Injection Risk	Higher risk if strings are concatenated.	Completely protected against SQL injection attacks.

11. SQLite vs Traditional Client-Server RDBMS (MySQL / Oracle)

Feature	SQLite Database	Traditional RDBMS (MySQL, PostgreSQL, Oracle)
Server Process	Serverless; embedded in application process.	Requires background client-server network daemon.
Administration	Zero-configuration required.	Requires user accounts, port configuration, and tuning.
Storage Model	Single cross-platform disk file.	Complex multi-file table spaces managed by server.
Concurrency	Single-writer locking at database level.	High-concurrency row-level locking for multi-user access.

12. Thread (Runnable) vs UI Main Thread

Feature	Main UI Thread	Background Worker Thread
Responsibility	Renders UI, dispatches touch events, manages lifecycle.	Executes heavy database queries, network calls, file I/O.
Blocking Risk	Blocking >5 seconds causes **ANR dialog**!	Runs asynchronously without freezing UI responsiveness.
UI Modification	Can modify View widgets directly.	Cannot modify Views directly (must use `runOnUiThread()` or Handler).

📖 25 Important One-Line Exam Definitions

2-Mark Quick Answers

Term	One-Line Exam-Ready Definition
1. Android OS	An open-source, Linux-based mobile operating system designed by Google and the Open Handset Alliance.
2. Open Handset Alliance (OHA)	A consortium of 84 telecom and hardware companies formed in 2007 to establish open mobile device standards.
3. Dalvik Virtual Machine (DVM)	A register-based Java virtual machine engineered for mobile devices to execute compact `.dex` bytecodes.
4. dx Tool	An Android SDK compiler tool that converts compiled Java `.class` files into a single `classes.dex` file.
5. Android Debug Bridge (ADB)	A command-line client-server bridge tool used to communicate, install APKs, and debug connected devices/emulators.
6. DDMS	Dalvik Debug Monitor Server — a profiling suite for inspecting process heaps, threads, and LogCat streams.
7. Encapsulation	The OOP technique of wrapping data fields and methods into a single class with restricted access via private modifiers.
8. Inheritance	Mechanism where a subclass inherits state and methods from a superclass using the `extends` keyword.
9. Polymorphism	The ability of a method or object to execute different behaviors at compile-time (overloading) or runtime (overriding).
10. Interface	A 100% abstract contract in Java containing method declarations implemented using `implements`.
11. Abstract Class	A class declared with `abstract` that cannot be instantiated and can contain both abstract and concrete methods.
12. Multithreading	Concurrent execution of multiple lightweight threads within a single Java process memory space.
13. JVM	Java Virtual Machine — an abstract computing machine executing Java bytecode (`.class`).

Term	One-Line Exam-Ready Definition
14. Garbage Collector	An automatic JVM memory manager that identifies and frees unreachable Heap memory objects.
15. AVD	Android Virtual Device — an emulator configuration defining hardware and API specifications on PC.
16. R.java	An auto-generated resource pointer class mapping all XML UI assets to unique 32-bit hexadecimal integer IDs.
17. AndroidManifest.xml	The root XML manifest registering an application's package, activities, permissions, and SDK versions.
18. Activity	A core Android component representing a single screen with an interactive user interface.
19. Activity Life Cycle	The sequence of 7 deterministic callback states (`onCreate` to `onDestroy`) an activity transitions through.
20. Intent	An asynchronous messaging object used to activate activities, services, or broadcast receivers.
21. Explicit Intent	An intent that explicitly specifies the exact target Java class to launch within an application.
22. Implicit Intent	An intent that specifies a general action leaving the OS to find matching applications via intent filters.
23. dp (Density-Independent Pixel)	A virtual unit scaled to a 160 dpi baseline screen used for responsive layout dimensions.
24. Toast	A transient feedback popup message that displays on screen briefly without stealing user input focus.
25. SQLiteOpenHelper	An abstract Android helper class that manages database creation (`onCreate`) and version upgrades (`onUpgrade`).

❓ Complete Expected Practice Question Bank

2M, 5M & 10M Questions

🟢 2-MARK SHORT QUESTIONS

1. Define Android Operating System and name its primary founders.
2. What is the Open Handset Alliance (OHA)?
3. What is Dalvik Virtual Machine (DVM)?
4. Explain the role of `R.java` in an Android project.
5. What is an Android Virtual Device (AVD)?
6. Define an Activity in Android.
7. What is an Intent? Mention its two types.
8. Differentiate `dp` and `sp` units in Android.
9. What is a Toast notification?
10. List 4 supported data types in SQLite database.
11. What is the purpose of `setContentView()` method?
12. Define Method Overloading and Method Overriding.

🟡 5-MARK MEDIUM QUESTIONS

1. Explain the key features of the Android Operating System.
2. Compare JVM and DVM with a neat compilation flowchart.
3. Explain 5 essential ADB (Android Debug Bridge) commands with syntax.
4. Explain the 4 Pillars of OOP in Java with brief code examples.
5. Compare Abstract Class and Interface in Java.
6. Explain Thread Life Cycle in Java with 5 states.
7. Describe the Android Project Directory Structure in Eclipse.
8. Explain Explicit Intent vs Implicit Intent with Java code.
9. Compare LinearLayout and RelativeLayout with key XML attributes.
10. Explain the role of `SQLiteOpenHelper` class and its methods.
11. Explain how to deploy an Android app to a real USB device.

🔴 10-MARK DETAILED QUESTIONS

1. Explain the 5-Layer Android Architecture in detail with a neat layered diagram.
2. Explain JVM Internal Architecture in detail with 5 runtime memory areas and execution engine.
3. Explain the complete Android Application Development Workflow in Eclipse with ADT from setup to USB deployment.
4. Explain the complete Activity Life Cycle in detail with 7 callbacks, state diagram, and real-world scenarios.
5. Explain SQLite database integration in Android and write complete Java code for Student CRUD operations.

Topper Answer-Writing Strategy & Time Allocation

Exam Mastery

How to Score 95%+ in Android Programming

29. **Standard Formal Definition First:** Always begin any answer with a 2-3 line formal definition box.
30. **Mandatory Visual Diagrams:** Always draw diagrams for:
    - Android 5-Layer Stack (Unit I)
    - JVM Internal Architecture (Unit II)
    - Compilation Workflow (javac $\to$ dx $\to$ DVM)
    - Activity 7-State Life Cycle (Unit IV)
    - App to SQLite Database Flow (Unit V)
31. **Complete Java Code Snippets:** For Unit II and Unit V, write complete classes with class name, methods, and expected outputs.
32. **Tabular Comparisons:** Always format differences in structured tables with distinct comparison parameters.
33. **Time Allocation Strategy:**
    - 2-Mark Questions: 2-3 minutes per question (One-line definition + 1 bullet point).
    - 5-Mark Questions: 7-8 minutes per question (Definition + 4 bullet points + small code/table).
    - 10-Mark Questions: 18-20 minutes per question (Introduction $\to$ Diagram $\to$ Layered Breakdown $\to$ Code $\to$ Conclusion).

ALL THE BEST FOR YOUR ANDROID PROGRAMMING EXAM! 🌟

Skill Enhancement Course (SEC)

Complete Exam Master & Topper Notes Package

Android Programming Notes Prepared & Curated by Aakash Rajawat

Connect & Follow on Professional Profiles:

Complete Android Programming Topper Notes Package

🌟 ALL THE BEST FOR YOUR ANDROID PROGRAMMING EXAM! 🌟

Android Programming Notes Prepared & Curated by Aakash Rajawat

Skill Enhancement Course (SEC)
Complete Exam Master & Topper Notes Package